

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..30.2

CREATE AN OAUTH2-BASED IDENTITY PROVIDER

 Copy as Markdown

Create an OAuth2-based identity provider

This page provides a complete, working implementation of a custom OAuth2 provider. The [GitHub provider](#) is built the same way, using the same OAuth2 utility shown here, so this example illustrates the general pattern for your own provider.

Overview

This example implements authentication with a fictional OAuth2 provider called "MyProvider". The implementation includes:

- Server-side token exchange and user management
- Client-side authorization flow
- Flutter UI integration
- Error handling

Server-side implementation

1. Data model

First, create a data model to store provider accounts:

```
class: MyProviderAccount
serverOnly: true
table: my_provider_account
fields:
  id: UuidValue?, defaultPersist=random_v7

# The AuthUser this account belongs to
authUser: module:serverpod_auth_core:AuthUser?, relation(onDelete=Cascade)
```



```

# Provider's user identifier
providerId: String

# User's email from provider (optional)
email: String?

# Creation timestamp
created: DateTime, defaultModel=now

indexes:
  my_provider_account_provider_id:
    fields: providerId
    unique: true

```

2. Configuration

Create the server configuration:

```

import 'package:serverpod_auth_idp_server/core.dart';
import 'my_provider_idp.dart';

class MyProviderIdpConfig extends IdentityProviderBuilder<MyProviderIdp> {
  final String clientId;
  final String clientSecret;
  late final OAuth2PkceServerConfig oauth2Config;

  MyProviderIdpConfig({
    required this.clientId,
    required this.clientSecret,
  }) : oauth2Config = OAuth2PkceServerConfig(
    tokenEndpointUrl: Uri.https('oauth.myprovider.com', '/oauth/token'),
    clientId: clientId,
    clientSecret: clientSecret,
    credentialsLocation: OAuth2CredentialsLocation.header,
    parseTokenResponse: parseTokenResponse,
  );

  static OAuth2PkceTokenResponse parseTokenResponse(Map<String, dynamic> response) {
    final error = response['error'] as String?;
    if (error != null) {
      final description = response['error_description'] as String?;
      throw OAuth2InvalidResponseException(
        'Provider error: $error${description != null ? ' - $description' : ''}',
      );
    }
    final token = response['access_token'] as String?;
    if (token == null) {
      throw const OAuth2MissingAccessTokenException('No access token in response');
    }
    return OAuth2PkceTokenResponse(accessToken: token);
  }
}

@override
MyProviderIdp build({

```

```

    required TokenManager tokenManager,
    required AuthUsers authUsers,
    required UserProfiles userProfiles,
  }) {
    return MyProviderIdp(
      config: this,
      tokenIssuer: tokenManager,
      authUsers: authUsers,
      userProfiles: userProfiles,
    );
  }
}

```

3. Provider class

Create the main identity provider class:

```

import 'dart:convert';

import 'package:http/http.dart' as http;
import 'package:serverpod/serverpod.dart';
import 'package:serverpod_auth_idp_server/core.dart';

import '../generated/protocol.dart';
import 'my_provider_idp_config.dart';

class MyProviderIdp implements IdentityProvider {
  /// Required by IdentityProvider: the stable method identifier for tokens.
  @override
  String get method ⇒ 'myprovider';

  /// Required by IdentityProvider: carry this provider's rows over when two
  /// auth users are merged.
  @override
  Future<void> mergeAuthUsers(
    Session session, {
    required UuidValue userToKeepId,
    required UuidValue userToRemoveId,
    required Transaction transaction,
  }) async {
    await MyProviderAccount.db.updateWhere(
      session,
      where: (t) ⇒ t.authUserId.equals(userToRemoveId),
      columnValues: (t) ⇒ [t.authUserId(userToKeepId)],
      transaction: transaction,
    );
  }

  final MyProviderIdpConfig config;
  final TokenIssuer _tokenIssuer;
  final AuthUsers _authUsers;
  final UserProfiles _userProfiles;

  late final OAuth2PkceUtil _oauth2Util;

```

```

MyProviderIdp({
  required this.config,
  required TokenIssuer tokenIssuer,
  required AuthUsers authUsers,
  required UserProfiles userProfiles,
}) : _tokenIssuer = tokenIssuer,
    _authUsers = authUsers,
    _userProfiles = userProfiles {
  _oauth2Util = OAuth2PkceUtil(config: config.oauth2Config);
}

Future<AuthSuccess> login(
  Session session, {
    required String code,
    required String codeVerifier,
    required String redirectUri,
  }) async {
  return await DatabaseUtil.runInTransactionOrSavepoint(
    session.db,
    null,
    (transaction) async {
      // 1. Exchange authorization code for token response
      final tokenResponse = await _oauth2Util.exchangeCodeForToken(
        code: code,
        codeVerifier: codeVerifier,
        redirectUri: redirectUri,
      );

      // 2. Fetch user information
      final userInfo = await _fetchUserInfo(session, tokenResponse.accessToken);

      // 3. Authenticate (find or create user)
      final account = await _authenticate(session, userInfo, transaction);

      // 4. Create user profile if new user
      if (account.newAccount) {
        await _createUserProfile(
          session,
          account.authUserId,
          userInfo,
          transaction,
        );
      }

      // 5. Issue authentication token
      return await _tokenIssuer.issueToken(
        session,
        authUserId: account.authUserId,
        transaction: transaction,
        method: method,
        scopes: account.scopes,
      );
    },
  );
}

Future<Map<String, dynamic>> _fetchUserInfo(
  Session session,

```

```

String accessToken,
) async {
  final response = await http.get(
    Uri.https('api.myprovider.com', '/v1/user'),
    headers: {
      'Authorization': 'Bearer $accessToken',
      'Accept': 'application/json',
    },
  );

  if (response.statusCode != 200) {
    session.log(
      'Failed to fetch user info: ${response.statusCode}',
      level: LogLevel.error,
    );
    throw MyProviderAuthException('Failed to fetch user information');
  }

  try {
    return jsonDecode(response.body) as Map<String, dynamic>;
  } catch (e) {
    session.log(
      'Failed to parse user info: $e',
      level: LogLevel.error,
    );
    throw MyProviderAuthException('Invalid user information format');
  }
}

Future<_AccountResult> _authenticate(
  Session session,
  Map<String, dynamic> userInfo,
  Transaction transaction,
) async {
  final providerId = userInfo['id']?.toString();
  if (providerId == null || providerId.isEmpty) {
    throw MyProviderAuthException('Missing user ID from provider');
  }

  // Check if account exists
  var account = await MyProviderAccount.db.findFirstRow(
    session,
    where: (t) => t.providerId.equals(providerId),
    transaction: transaction,
  );

  final isNewAccount = account == null;

  if (isNewAccount) {
    // Create new auth user
    final authUser = await _authUsers.create(
      session,
      transaction: transaction,
    );

    // Create provider account
    account = await MyProviderAccount.db.insertRow(
      session,

```

```

        MyProviderAccount(
            providerId: providerId,
            email: userInfo['email'] as String?,
            authUserId: authUser.id,
        ),
        transaction: transaction,
    );

    return (
        authUserId: authUser.id,
        newAccount: true,
        scopes: authUser.scopes,
    );
} else {
    // Get existing user
    final authUser = await _authUsers.get(
        session,
        authUserId: account.authUserId,
        transaction: transaction,
    );

    return (
        authUserId: authUser.id,
        newAccount: false,
        scopes: authUser.scopes,
    );
}
}

Future<void> _createUserProfile(
    Session session,
    UuidValue authUserId,
    Map<String, dynamic> userInfo,
    Transaction transaction,
) async {
    try {
        await _userProfiles.createUserProfile(
            session,
            authUserId,
            UserProfileData(
                fullName: userInfo['name'] as String?,
                email: userInfo['email'] as String?,
            ),
            transaction: transaction,
        );
    } catch (e, stackTrace) {
        session.log(
            'Failed to create user profile',
            level: LogLevel.error,
            exception: e,
            stackTrace: stackTrace,
        );
        // Don't fail the authentication if profile creation fails
    }
}

typedef _AccountResult = ({

```

```

    UuidValue authUserId,
    bool newAccount,
    Set<Scope> scopes,
  });

class MyProviderAuthException implements Exception {
  final String message;
  const MyProviderAuthException(this.message);

  @override
  String toString() => 'MyProviderAuthException: $message';
}

```

4. Endpoint

Create the endpoint:

```

import 'package:serverpod/serverpod.dart';
import 'package:serverpod_auth_idp_server/core.dart';

import '../generated/protocol.dart';
import 'my_provider_idp.dart';

class MyProviderIdpEndpoint extends IdpBaseEndpoint {
  MyProviderIdp get myProviderIdp =>
    AuthServices.getIdentityProvider<MyProviderIdp>();

  /// Required by IdpBaseEndpoint: report whether the signed-in user already
  /// has an account with this provider.
  @override
  Future<bool> hasAccount(Session session) async {
    final authUserId = session.authenticated?.authUserId;
    if (authUserId == null) return false;
    return await MyProviderAccount.db.findFirstRow(
      session,
      where: (t) => t.authUserId.equals(authUserId),
    ) !=
      null;
  }

  Future<AuthSuccess> login(
    Session session, {
    required String code,
    required String codeVerifier,
    required String redirectUri,
  }) async {
    try {
      return await myProviderIdp.login(
        session,
        code: code,
        codeVerifier: codeVerifier,
        redirectUri: redirectUri,
      );
    } on OAuth2Exception catch (e) {
      session.log(

```

```

        'OAuth2 error during authentication: ${e.message}',
        level: LogLevel.error,
    );
    throw Exception('Authentication failed');
} on MyProviderAuthException catch (e) {
    session.log(
        'MyProvider error: ${e.message}',
        level: LogLevel.error,
    );
    throw Exception('Authentication failed');
}
}
}
}

```

5. Server registration

Register the provider in `server.dart`:

```

import 'package:serverpod_auth_idp_server/core.dart';

import 'src/generated/serverpod.dart';
import 'my_provider_idp_config.dart';

void run(List<String> args) async {
    final pod = Serverpod(args);

    final myProviderConfig = MyProviderIdpConfig(
        clientId: pod.getPassword('myProviderClientId')!,
        clientSecret: pod.getPassword('myProviderClientSecret')!,
    );

    pod.initializeAuthServices(
        tokenManagerBuilders: [
            JwtConfigFromPasswords(),
        ],
        identityProviderBuilders: [
            myProviderConfig,
        ],
    );

    await pod.start();
}

```

Client-side implementation

1. Configuration

Create the client configuration:

```

import 'package:flutter/foundation.dart';
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

```



```

class MyProviderConfig {
  static const _clientIdEnvKey = 'MY_PROVIDER_CLIENT_ID';
  static const _redirectUriEnvKey = 'MY_PROVIDER_REDIRECT_URI';

  static OAuth2PkceProviderClientConfig get clientConfig {
    // Get credentials from environment or use defaults
    final clientId = _getClientId();
    final redirectUri = _getRedirectUri();

    return OAuth2PkceProviderClientConfig(
      authorizationEndpoint: Uri.https('oauth.myprovider.com', '/oauth/authorize'),
      clientId: clientId,
      redirectUri: redirectUri,
      callbackUrlScheme: Uri.parse(redirectUri).scheme,
      defaultScopes: ['profile', 'email'],
    );
  }

  static String _getClientId() {
    const clientId = String.fromEnvironment(_clientIdEnvKey);
    if (clientId.isNotEmpty) return clientId;

    // Development fallback
    if (kDebugMode) {
      return 'dev-client-id';
    }

    throw Exception('$_clientIdEnvKey not configured');
  }

  static String _getRedirectUri() {
    const redirectUri = String.fromEnvironment(_redirectUriEnvKey);
    if (redirectUri.isNotEmpty) return redirectUri;

    // Platform-specific defaults for development
    if (kDebugMode) {
      if (kIsWeb) {
        return 'http://localhost:3000/auth.html';
      } else {
        return 'myapp://auth-callback';
      }
    }

    throw Exception('$_redirectUriEnvKey not configured');
  }
}

```

2. Service

Create the sign-in service:

```

import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';

import 'my_provider_config.dart';

```

```

class MyProviderService {
  static final instance = MyProviderService._();
  MyProviderService._();

  OAuth2PkceUtil? _oauth2Util;

  void initialize() {
    if (_oauth2Util != null) return;

    _oauth2Util = OAuth2PkceUtil(
      config: MyProviderConfig.clientConfig,
    );
  }

  Future<OAuth2PkceResult> signIn({List<String>? scopes}) async {
    if (_oauth2Util == null) {
      throw StateError(
        'MyProviderService not initialized. Call initialize() first.',
      );
    }

    return await _oauth2Util!.authorize(scopes: scopes);
  }
}

```

3. Controller

Create an authentication controller:

```

import 'package:flutter/foundation.dart';
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';
import 'package:your_client/your_client.dart';

import 'my_provider_config.dart';
import 'my_provider_service.dart';

enum MyProviderAuthState {
  idle,
  loading,
  authenticated,
  error,
}

class MyProviderAuthController extends ChangeNotifier {
  final Client client;
  final VoidCallback? onAuthenticated;
  final Function(Object error)? onError;

  MyProviderAuthController({
    required this.client,
    this.onAuthenticated,
    this.onError,
  });

  MyProviderAuthState _state = MyProviderAuthState.idle;

```

```

Object? _error;

MyProviderAuthState get state => _state;
bool get isLoading => _state == MyProviderAuthState.loading;
bool get isAuthenticated => client.auth.isAuthenticated;
String? get errorMessage => _error?.toString();

Future<void> signIn() async {
  if (_state == MyProviderAuthState.loading) return;

  _setState(MyProviderAuthState.loading);

  try {
    // Get authorization code from provider
    final result = await MyProviderService.instance.signIn();

    // Exchange for tokens on the server. The generated client names the
    // endpoint after the class, with the Endpoint suffix dropped, so
    // MyProviderIdpEndpoint becomes client.myProviderIdp.
    final authSuccess = await client.myProviderIdp.login(
      code: result.code,
      // Safe to unwrap: this config keeps PKCE enabled, so the
      // verifier is always present.
      codeVerifier: result.codeVerifier!,
      redirectUri: MyProviderConfig.clientConfig.redirectUri,
    );

    // Register the session, otherwise the app is never actually signed in.
    await client.auth.updateSignedInUser(authSuccess);

    _setState(MyProviderAuthState.authenticated);
    onAuthenticated?.call();
  } catch (error) {
    // A cancelled sign-in arrives as OAuth2PkceUnknownException.
    _error = error;
    _setState(MyProviderAuthState.error);
    onError?.call(error);
  }
}

void _setState(MyProviderAuthState newState) {
  if (newState != MyProviderAuthState.error) {
    _error = null;
  }
  _state = newState;
  notifyListeners();
}
}

```

4. UI widget

Create the sign-in button:

```

import 'package:flutter/material.dart';
import 'package:your_client/your_client.dart';

```

```

import 'my_provider_auth_controller.dart';

class MyProviderSignInWidget extends StatefulWidget {
  final Client client;
  final VoidCallback? onAuthenticated;
  final Function(Object error)? onError;

  const MyProviderSignInWidget({
    required this.client,
    this.onAuthenticated,
    this.onError,
    super.key,
  });

  @override
  State<MyProviderSignInWidget> createState() => _MyProviderSignInWidgetState();
}

class _MyProviderSignInWidgetState extends State<MyProviderSignInWidget> {
  late final MyProviderAuthController _controller;

  @override
  void initState() {
    super.initState();
    _controller = MyProviderAuthController(
      client: widget.client,
      onAuthenticated: widget.onAuthenticated,
      onError: widget.onError,
    );
    _controller.addListener(_onControllerStateChanged);
  }

  @override
  void dispose() {
    _controller.removeListener(_onControllerStateChanged);
    _controller.dispose();
    super.dispose();
  }

  void _onControllerStateChanged() => setState(() {});

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: _controller.isLoading ? null : _controller.signIn,
      style: ElevatedButton.styleFrom(
        minimumSize: const Size(240, 48),
      ),
      child: _controller.isLoading
        ? const SizedBox(
            width: 20,
            height: 20,
            child: CircularProgressIndicator(strokeWidth: 2),
          )
        : const Text('Sign in with MyProvider'),
    );
  }
}

```

```
}
```

5. Initialization

Initialize in your app:

```
import 'package:flutter/material.dart';
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';
import 'package:serverpod_flutter/serverpod_flutter.dart';
import 'package:your_client/your_client.dart';

import 'my_provider_service.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Initialize Serverpod client
  final client = Client('http://localhost:8080/')
    ..connectivityMonitor = FlutterConnectivityMonitor()
    ..authSessionManager = FlutterAuthSessionManager();

  await client.auth.initialize();

  // Initialize MyProvider service
  MyProviderService.instance.initialize();

  runApp(MyApp(client: client));
}
```

6. Usage in UI

Use the widget in your sign-in page:

```
import 'package:flutter/material.dart';
import 'package:your_client/your_client.dart';

import 'my_provider_sign_in_widget.dart';

class SignInPage extends StatelessWidget {
  final Client client;

  const SignInPage({required this.client, super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Sign In')),
      body: Center(
        child: MyProviderSignInWidget(
          client: client,
          onAuthenticated: () {
            // Navigate to home page after authentication
            Navigator.of(context).pushReplacement(
```

```

        MaterialPageRoute(
          builder: (_) => HomePage(client: client),
        ),
      ),
    },
    onError: (error) {
      // Show error message
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Sign in failed: $error'),
          backgroundColor: Colors.red,
        ),
      ),
    },
  },
),
);
}
}

```

This compact example provides a clear template you can adapt to integrate any OAuth2 provider into your Serverpod project. Use this as a template for implementing your own OAuth2 provider integration.